VHDL Assignment-3

학번: 2024043545
이름: 김예준

Problem 1. RS Latch

Requirements

두 개의 NOR 게이트를 교차 연결하여 RS Latch를 구성한다. VHDL에서도 같은 피드백 구조를 구현하고 Set, Reset, Hold 및 금지 입력 조건을 확인한다.

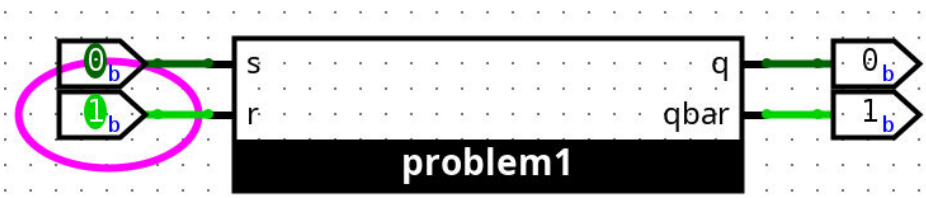
입력: s, r / 출력: q, qbar. 모든 입출력은 1비트이다.

1. VHDL code

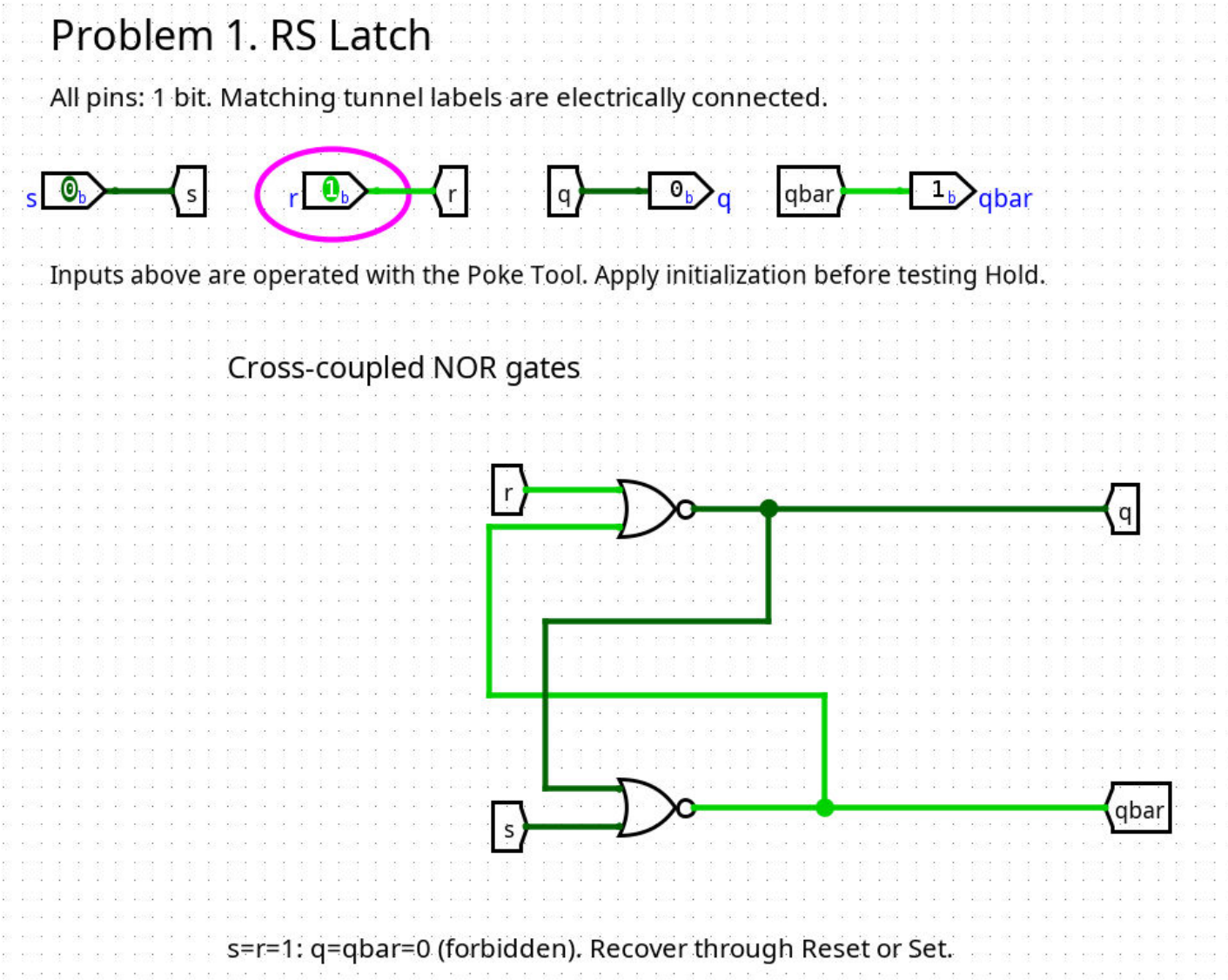
```
library ieee;
use ieee.std_logic_1164.all;
entity problem1 is
    port (
        s, r : in std_logic;
        q, qbar : out std_logic
    );
end problem1;
architecture structural of problem1 is
    signal qi, qbi : std_logic;
begin
    qi <= not (r or qbi);
    qbi <= not (s or qi);
    q <= qi;
    qbar <= qbi;
end structural;
```

2. Basic circuit

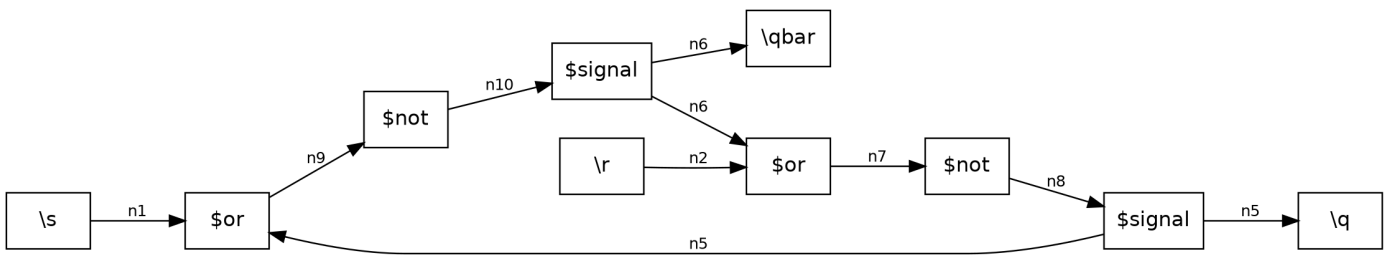
main에서 s와 r을 problem1의 입력에 연결하고 q와 qbar를 출력 핀에 연결하였다. 모든 핀은 1비트이다. 시작할 때 s=0, r=1로 Reset한 뒤 시험하였다.



NOR 게이트 두 개의 출력을 서로 반대쪽 입력에 연결하였다. 위쪽 게이트는 r과 qbar를 받아 q를 만들고, 아래쪽 게이트는 s와 q를 받아 qbar를 만든다.



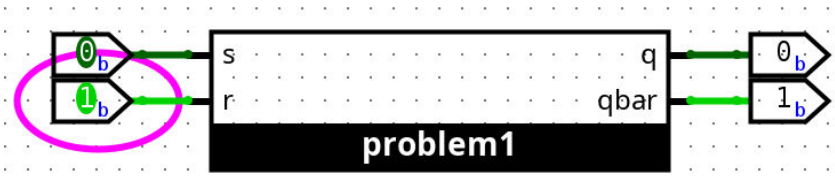
GHDL로 논리 합성을 수행하였다. 생성 회로도에서 OR와 NOT의 조합으로 된 두 NOR 경로와 교차 피드백을 확인하였다. 이 회로의 피드백은 상태를 저장하기 위한 의도적인 구조이다.



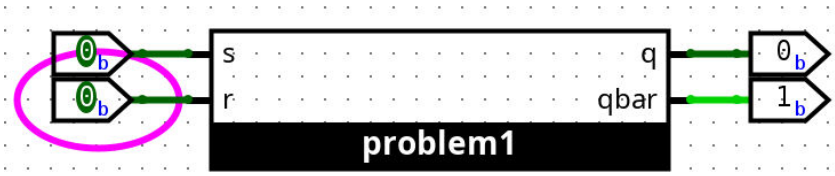
3. Simulation results

Logisim-evolution 5.0.0과 GHDL 5.0.1을 사용하였다. main의 VHDL 블록은 GHDL 연결 프로그램으로 실제 코드를 실행하여 확인하였고, 게이트 회로는 Logisim에서 따로 검사하였다. 아래 사진은 VHDL 블록의 실행 화면이다.

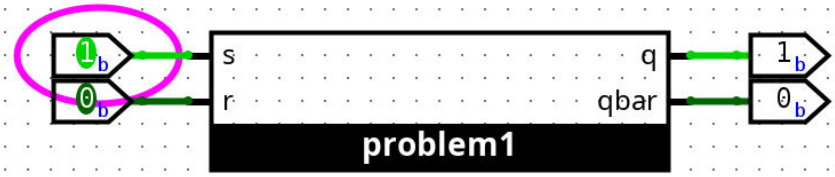
Reset, Hold, Set, Hold, 금지 입력의 순서로 확인하였다. 금지 입력에서는 두 출력이 모두 0이 되어 상보 관계가 깨진다. 복구할 때에는 s만 먼저 0으로 내려 Reset 상태를 만들었다.



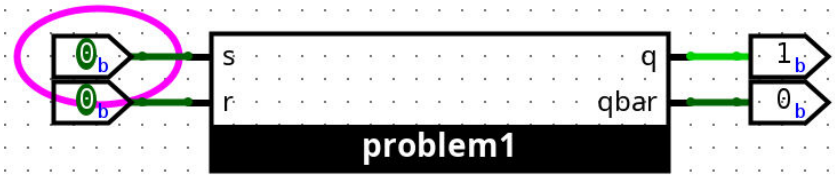
s=0, r=1에서 q=0으로 Reset하였다.



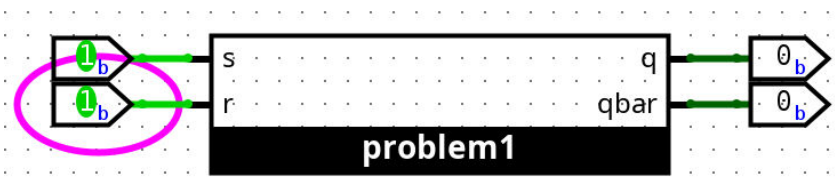
s=r=0으로 바꾼 뒤 0이 유지되었다.



s=1, r=0에서 q=1로 Set되었다.

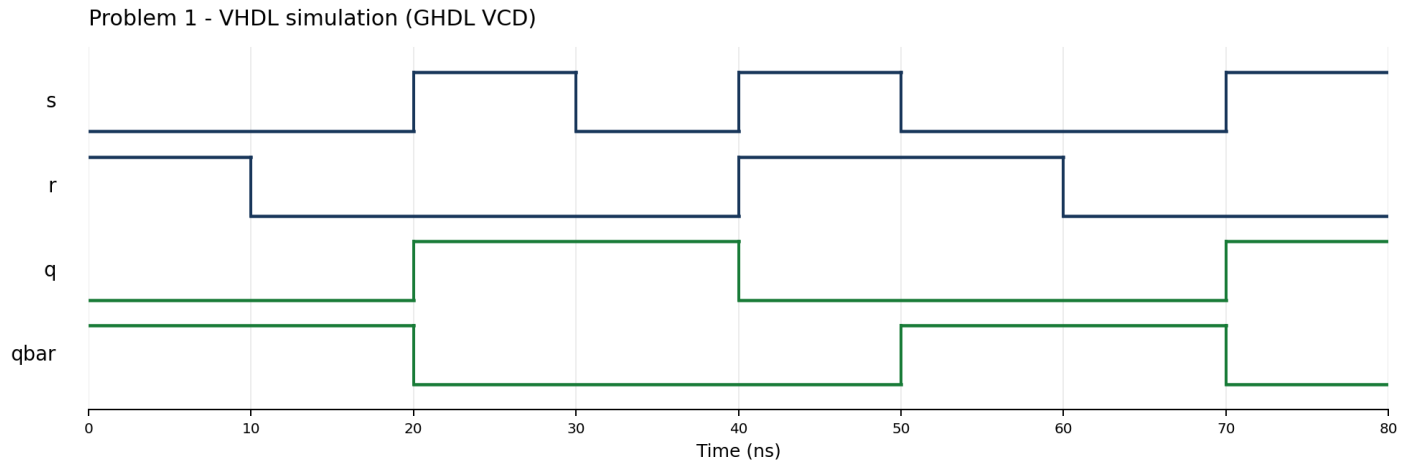


s=r=0으로 바꾼 뒤 1이 유지되었다.



s=r=1에서 q=qbar=0이 되었다.

0~10 ns에서 Reset으로 q를 0으로 초기화하였다. 20 ns에 Set을 주면 q가 1이 되고, 30 ns에 두 입력을 0으로 내려도 값이 유지되었다. 40~50 ns의 금지 입력에서는 q와 qbar가 모두 0이었다. 이후 s만 0으로 내려 Reset 상태를 거쳐 복구하였다.



파형은 GHDL이 기록한 VCD를 표시한 것이다. 각 입력 단계는 10 ns이며, 게이트 회로에도 같은 순서의 입력을 적용하였다.

4. RESULTS

목적
NOR 게이트의 피드백을 이용한 1비트 기억 동작을 확인한다.

회로 구성
NOR 게이트 2개와 1비트 입력 핀 2개, 출력 핀 2개로 구성하였다. VHDL도 두 개의 동시 신호 할당문으로 같은 구조를 표현하였다.

동작 설명
s=1, r=0이면 q가 1이 되고, s=0, r=1이면 q가 0이 된다. s=r=0에서는 피드백에 의해 이전 상태를 유지한다. s=r=1일 때에는 두 출력이 모두 0이므로 정상적인 상보 관계가 깨진다.

8개 검사에서 Logisim, VHDL 및 합성 netlist의 결과가 일치하였다. 금지 입력을 해제할 때 두 입력을 동시에 0으로 내리는 경우의 최종 상태는 보장할 수 없으므로, Reset을 먼저 거쳐 복구하였다.

Problem 2. D Latch

Requirements

Enable 입력 en을 갖는 D Latch를 VHDL과 NAND 게이트로 구현한다. en=1일 때 q가 d를 따라가고, en=0일 때 입력 d가 변해도 이전 출력을 유지하는지 확인한다.

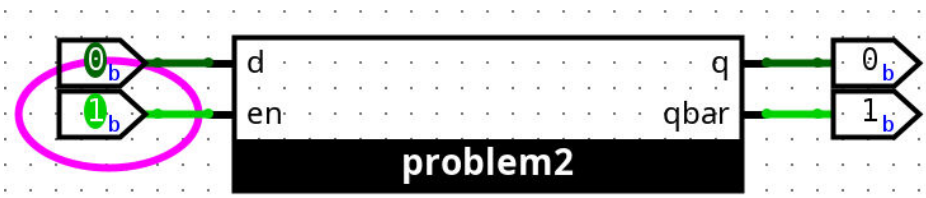
입력: d, en / 출력: q, qbar. 모든 입출력은 1비트이다.

1. VHDL code

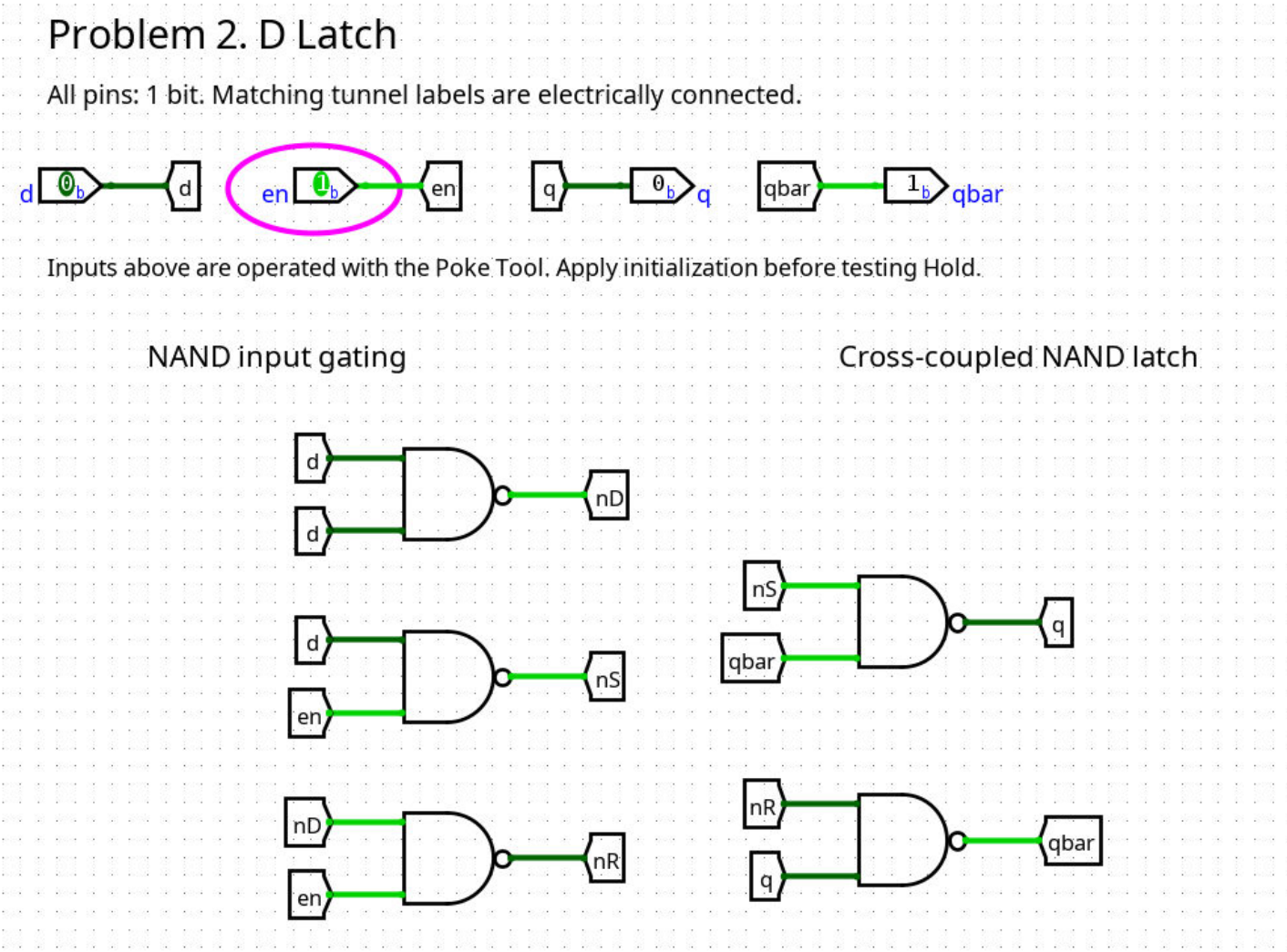
```
library ieee;
use ieee.std_logic_1164.all;
entity problem2 is
    port (
        d, en : in std_logic;
        q, qbar : out std_logic
    );
end problem2;
architecture behavioral of problem2 is
    signal qi : std_logic;
begin
    process(d, en)
    begin
        if en = '1' then
            qi <= d;
        end if;
    end process;
    q <= qi;
    qbar <= not qi;
end behavioral;
```

2. Basic circuit

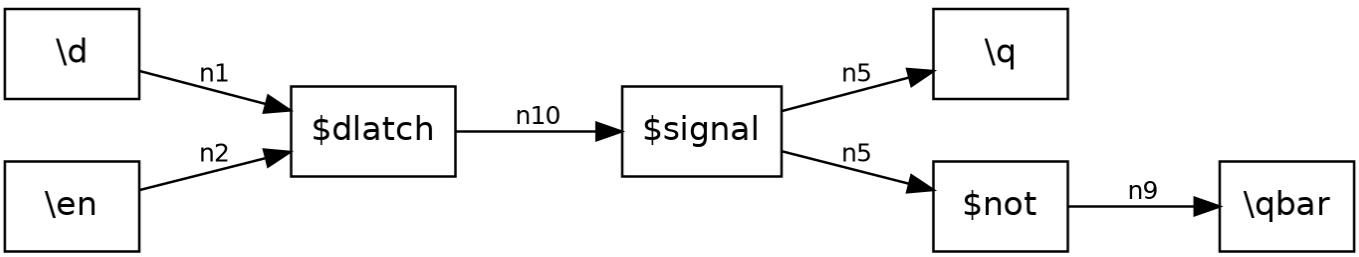
main에서 d와 en을 problem2의 입력에 연결하였다. q와 qbar는 각각 1비트 출력 핀에 연결하였다. 처음에는 d=0, en=1로 저장값을 정한 뒤 입력을 변경하였다.



NAND 게이트 5개를 사용하였다. 첫 번째 NAND의 두 입력에 d를 연결하여 반전 신호 nD를 만들었다. d와 en, nD와 en으로 nS와 nR을 만든 뒤 교차 연결된 두 NAND에 입력하였다.

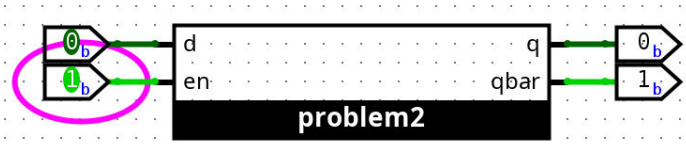


GHDL 합성 회로도에 $dlatch가 생성되었다. en에 따른 저장 동작이 래치로 표현되며, qbar는 q를 반전한 값이다. 생성된 VHDL netlist에도 같은 시험 입력을 적용하였다.

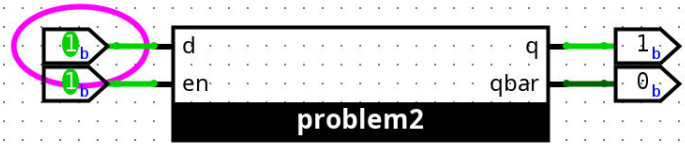


3. Simulation results

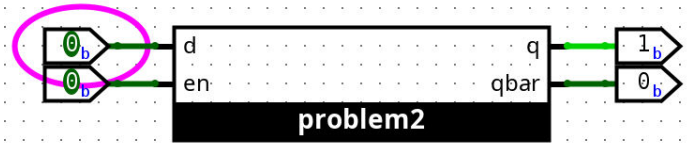
en=1일 때 d가 출력에 전달되는지 확인하고, 1과 0을 각각 저장한 뒤 en=0에서 d를 바꾸어 유지 동작을 확인하였다.



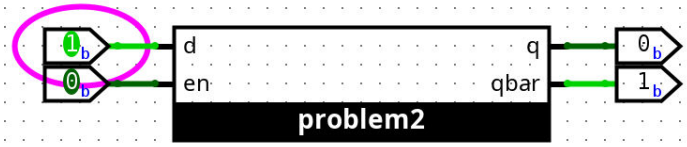
en=1, d=0에서 q=0이 되었다.



en=1, d=1에서 q=1이 되었다.

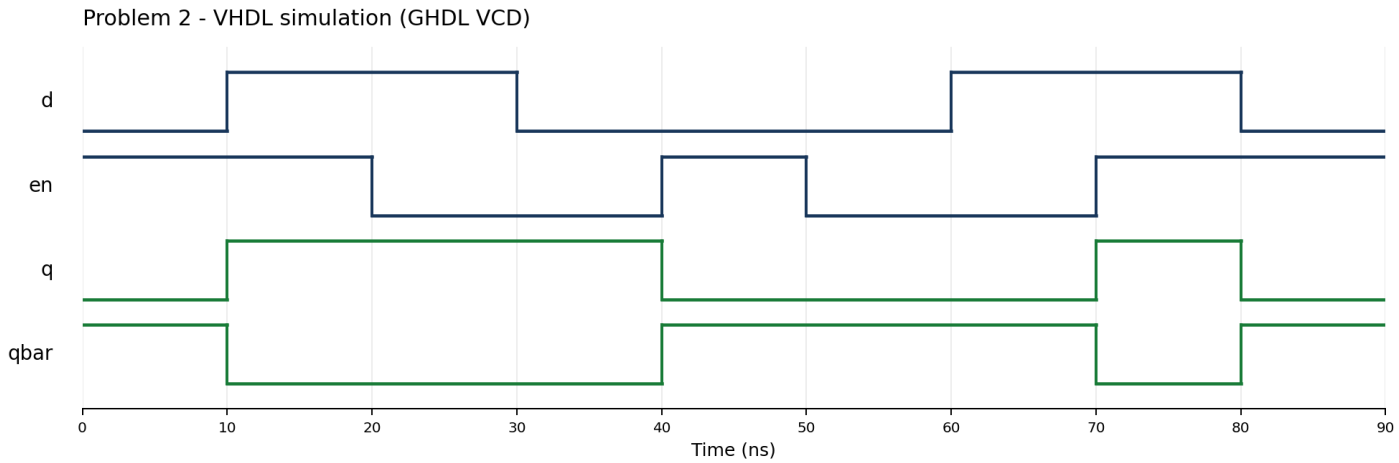


en=0에서 d를 0으로 바꾸어도 q=1을 유지하였다.



0을 저장한 뒤 en=0에서 d를 1로 바꾸어도 q=0이었다.

en=1인 0~20 ns에서 d의 변화에 따라 q가 변하였다. 20 ns에 en을 0으로 내린 후 30 ns에 d를 0으로 바꾸어도 q는 1을 유지하였다. 반대로 50~70 ns에서는 d가 1로 변해도 저장된 0이 유지되었다.



파형은 GHDL이 기록한 VCD를 표시한 것이다. 각 입력 단계는 10 ns이며, 게이트 회로에도 같은 순서의 입력을 적용하였다.

4. RESULTS

목적
Enable 신호에 따라 입력을 전달하거나 이전 상태를 저장하는 D Latch의 동작을 확인한다.

회로 구성
입력 반전용 NAND 1개, 입력 제어용 NAND 2개, 상태 저장용 NAND 2개로 구성하였다. VHDL에서는 en=1일 때만 qi에 d를 할당하였다.

동작 설명
en=1이면 nS와 nR이 d에 따라 변하므로 q가 d를 따라간다. en=0이면 nS=nR=1이 되어 교차 연결된 NAND가 이전 상태를 유지한다. VHDL에서 en=0일 때 새 값을 할당하지 않은 것도 같은 저장 동작을 나타낸다.

9개 검사에서 세 구현의 결과가 일치하였다. 1을 저장한 경우와 0을 저장한 경우를 각각 확인하였고, Enable이 꺼진 동안 d의 변화가 출력에 전달되지 않았다.

Problem 3. D Flip-Flop

Requirements

rising_edge(clk)를 사용하여 상승 에지에서 동작하는 D Flip-Flop을 구현한다. Logisim에서는 NAND 게이트로 만든 두 D Latch와 인버터를 연결하여 Master-Slave 구조를 구성한다.

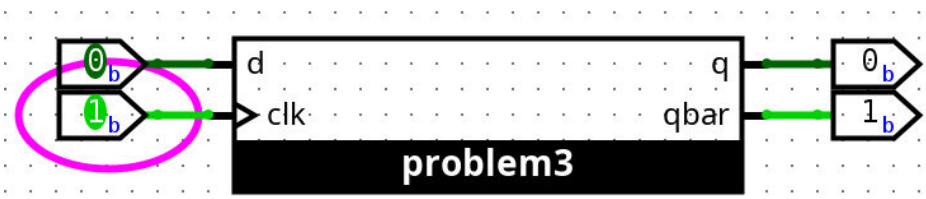
입력: d, clk / 출력: q, qbar. clk는 1비트 입력 핀으로 직접 0과 1을 변경하였다.

1. VHDL code

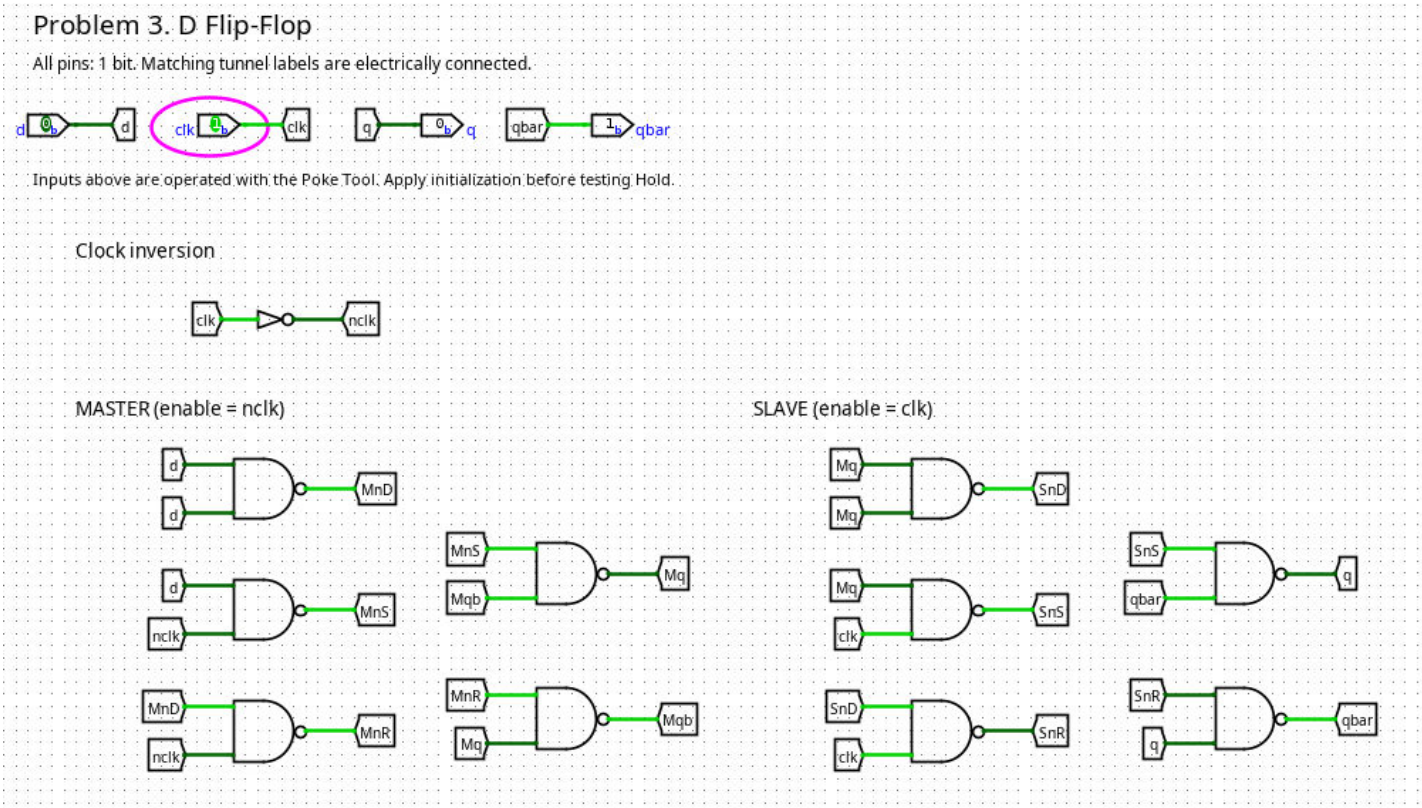
```
library ieee;
use ieee.std_logic_1164.all;
entity problem3 is
    port (
        d, clk : in std_logic;
        q, qbar : out std_logic
    );
end problem3;
architecture behavioral of problem3 is
    signal qi : std_logic;
begin
    process(clk)
    begin
        if rising_edge(clk) then
            qi <= d;
        end if;
    end process;
    q <= qi;
    qbar <= not qi;
end behavioral;
```

2. Basic circuit

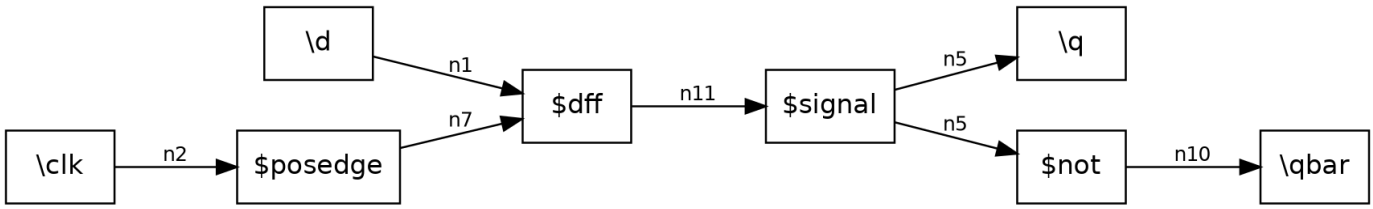
main에서 d와 clk를 problem3에 연결하고 q, qbar를 출력 핀에 연결하였다. 모든 핀은 1비트이다. d=0, clk=0에서 시작하여 clk를 1로 올린 뒤 결과를 확인하였다.



Master의 Enable에는 NOT clk, Slave의 Enable에는 clk를 연결하였다. Master의 출력 Mq가 Slave의 데이터 입력이 되고, Slave의 출력이 최종 q가 된다. 두 래치는 각각 NAND 게이트 5개로 구성하였다.

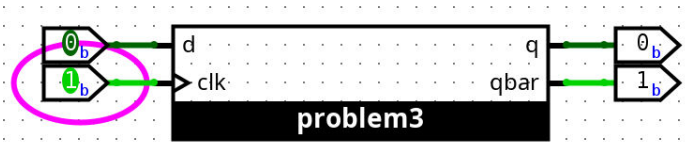


GHDL 합성 회로도에 상승 에지 D Flip-Flop을 나타내는 $dff가 생성되었다. 래치 두 개로 구성한 Logisim 회로와 저장 기능은 같으며, 합성 회로에서는 하나의 플립플롭으로 표현된다.

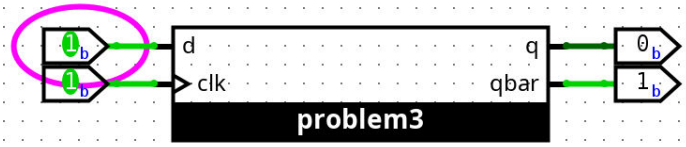


3. Simulation results

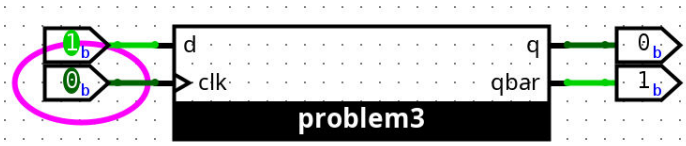
클록이 1로 유지되는 구간과 하강 에지에서는 값이 바뀌지 않고, 상승 에지에서만 d가 q로 전달되는지 확인하였다.



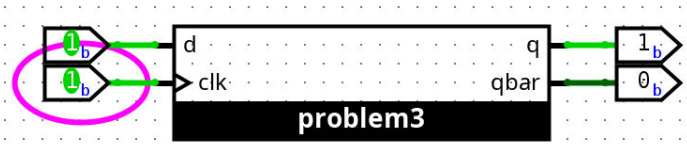
첫 상승 에지에서 d=0을 저장하였다.



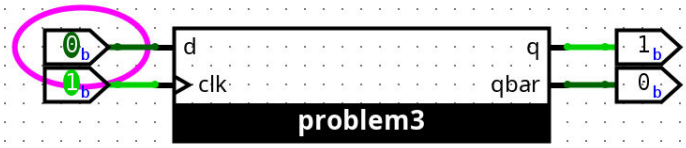
clk=1에서 d를 바꾸어도 q=0이었다.



하강 에지에서도 q=0을 유지하였다.

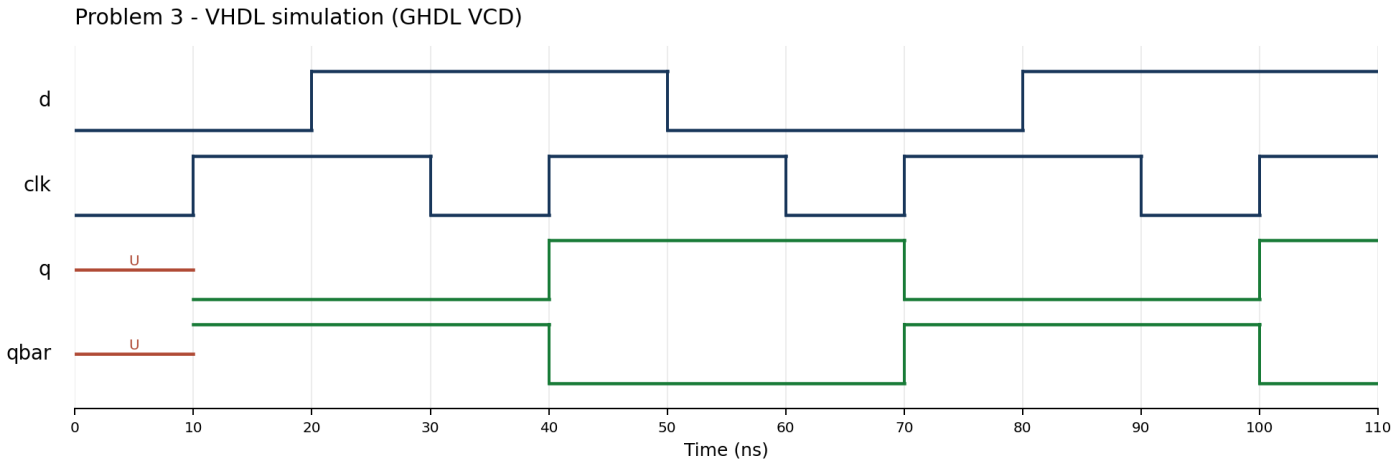


다음 상승 에지에서 d=1을 저장하였다.



clk=1에서 d=0으로 바꾸어도 q=1이었다.

10 ns의 첫 상승 에지에서 d=0을 저장하였다. 20 ns에 clk=1인 상태에서 d를 1로 바꾸어도 q는 변하지 않았다. 30 ns의 하강 에지에서도 유지되었고, 40 ns의 상승 에지에서 q=1이 되었다. 70 ns에는 d=0을 저장하였다.



파형은 GHDL이 기록한 VCD를 표시한 것이다. 각 입력 단계는 10 ns이며, 게이트 회로에도 같은 순서의 입력을 적용하였다.

4. RESULTS

목적
레벨에 반응하는 D Latch와 상승 에지에 반응하는 D Flip-Flop의 차이를 확인한다.

회로 구성
NAND 게이트 10개와 NOT 게이트 1개로 두 래치와 반전 클록을 구성하였다. 미리 만들어진 Latch나 Flip-Flop 부품은 사용하지 않았다.

동작 설명
clk=0일 때 Master가 입력을 받아들이고 Slave는 이전 출력을 유지한다. clk가 1로 바뀌면 Master가 닫히고 Slave가 열려 Master에 저장된 값이 출력된다. clk=1 동안 d가 바뀌거나 하강 에지가 발생하여도 최종 q는 유지된다.

초기화 단계 다음의 10개 검사에서 세 구현의 결과가 일치하였다. 상승 에지에서의 0과 1 저장, 높은 클록 레벨에서의 입력 변화, 하강 에지에서의 유지 동작을 확인하였다.

Problem 4. JK Flip-Flop

Requirements

상승 에지에서 동작하는 JK Flip-Flop을 구현한다. NAND 게이트와 출력 피드백을 이용하여 Hold, Reset, Set, Toggle의 네 가지 모드를 확인한다.

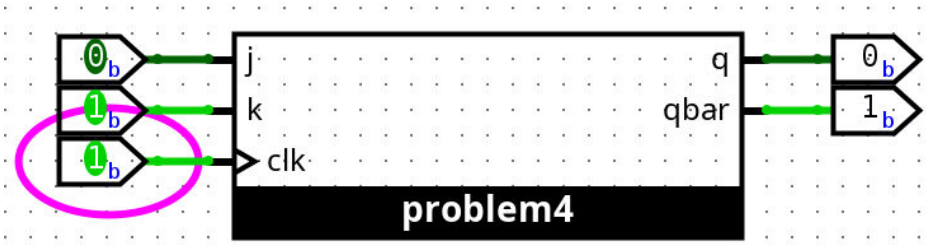
입력: j, k, clk / 출력: q, qbar. j=k=1에서는 상승 에지마다 출력이 한 번씩 반전되어야 한다.

1. VHDL code

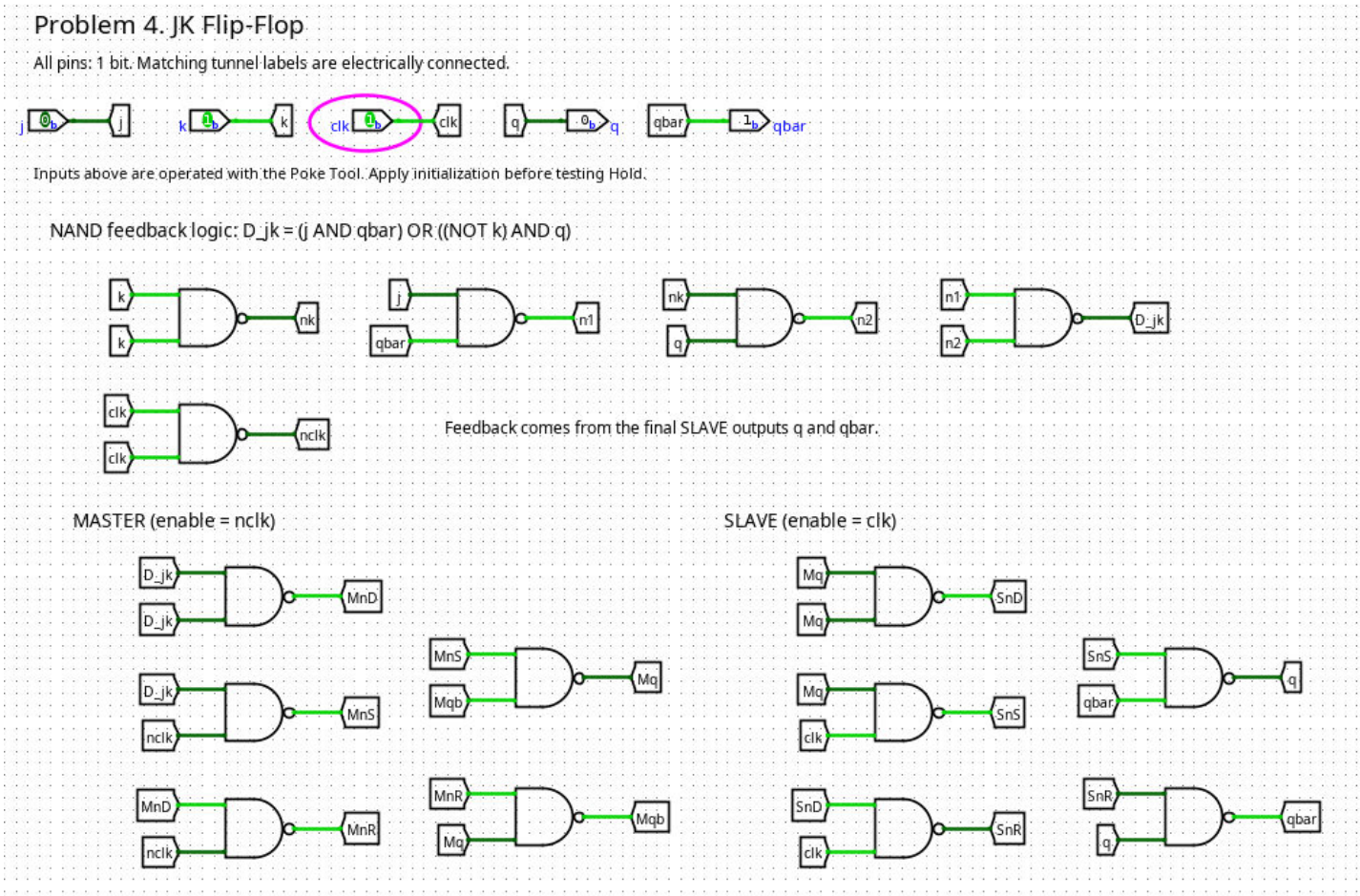
```
library ieee;
use ieee.std_logic_1164.all;
entity problem4 is
    port (
        j, k, clk : in std_logic;
        q, qbar : out std_logic
    );
end problem4;
architecture behavioral of problem4 is
    signal qi : std_logic;
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if j = '0' and k = '0' then
                null;
            elsif j = '0' and k = '1' then
                qi <= '0';
            elsif j = '1' and k = '0' then
                qi <= '1';
            elsif j = '1' and k = '1' then
                qi <= not qi;
            end if;
        end if;
    end process;
    q <= qi;
    qbar <= not qi;
end behavioral;
```

2. Basic circuit

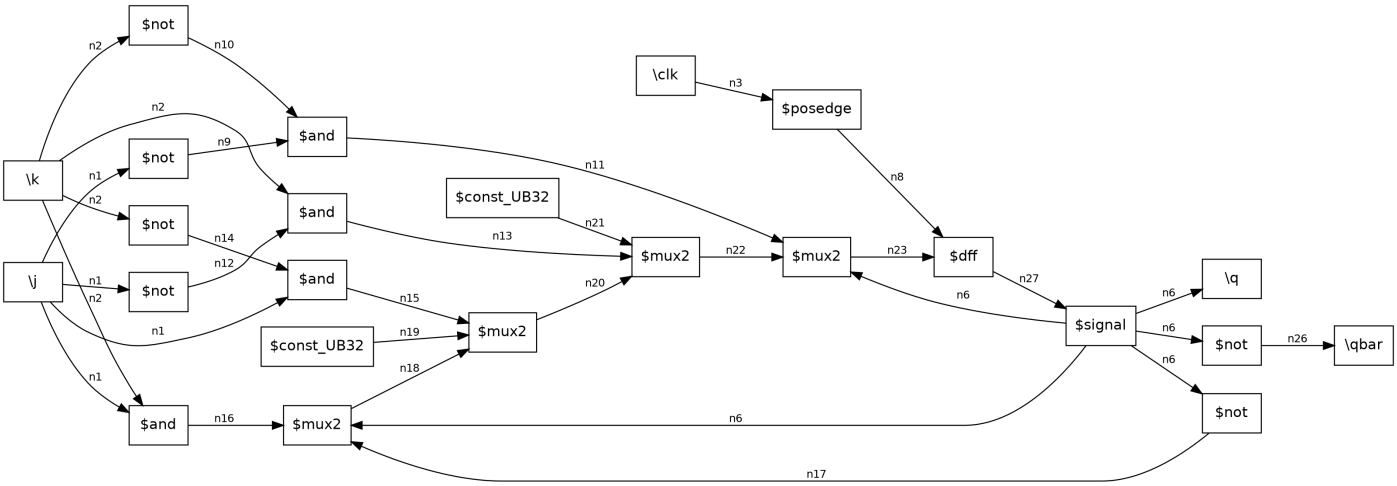
main에서 j, k, clk를 problem4에 연결하였다. 모든 핀은 1비트이다. 처음에는 j=0, k=1, clk=0으로 설정하고 clk를 1로 올려 q=0으로 초기화하였다. 이후에는 clk=0에서 j와 k를 정한 다음 상승 에지를 주었다.



JK의 다음 상태 식 D_jk=(j AND qbar) OR ((NOT k) AND q)를 NAND 게이트 4개로 구성하였다. 이 값을 NAND 게이트로 만든 Master-Slave D Flip-Flop에 입력하였다. 피드백에는 최종 Slave 출력 q와 qbar를 사용하였다.

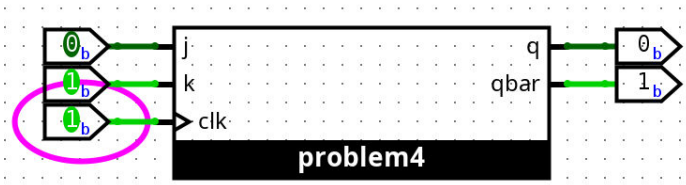


VHDL을 합성하면 JK 동작을 위한 선택 논리와 상태 저장 소자가 생성된다. 합성 회로도에 나타난 논리는 현재 q와 j, k로 다음 값을 결정한다. 합성 netlist에 대해서도 Hold, Set, Reset, Toggle을 다시 검사하였다.

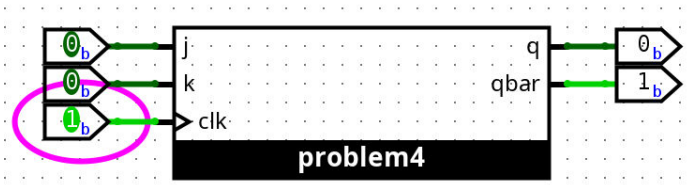


3. Simulation results

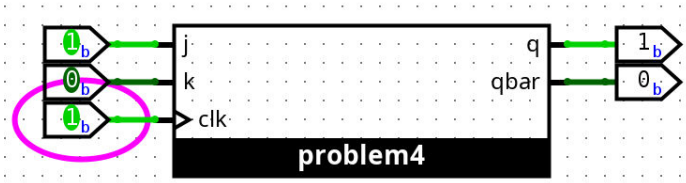
Reset으로 초기화한 뒤 Hold, Set과 Toggle을 확인하였다. j=k=1에서는 연속된 두 상승 에지마다 출력이 한 번씩 반전되었다. clk를 1로 유지할 때에는 추가 반전이 없었다.



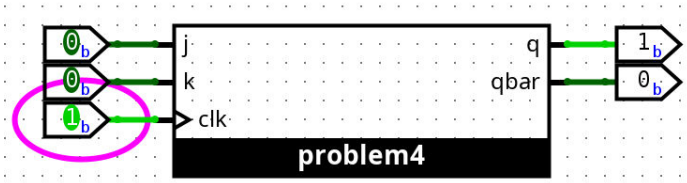
j=0, k=1의 상승 에지에서 q=0이 되었다.



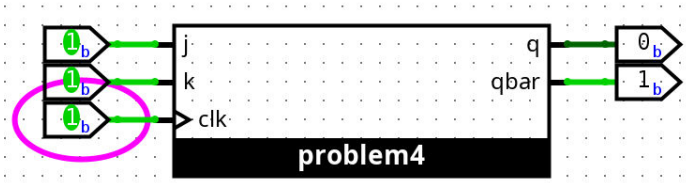
j=k=0의 상승 에지에서 0을 유지하였다.



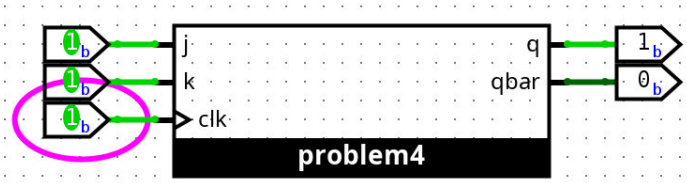
j=1, k=0의 상승 에지에서 q=1이었다.



j=k=0의 상승 에지에서 1을 유지하였다.

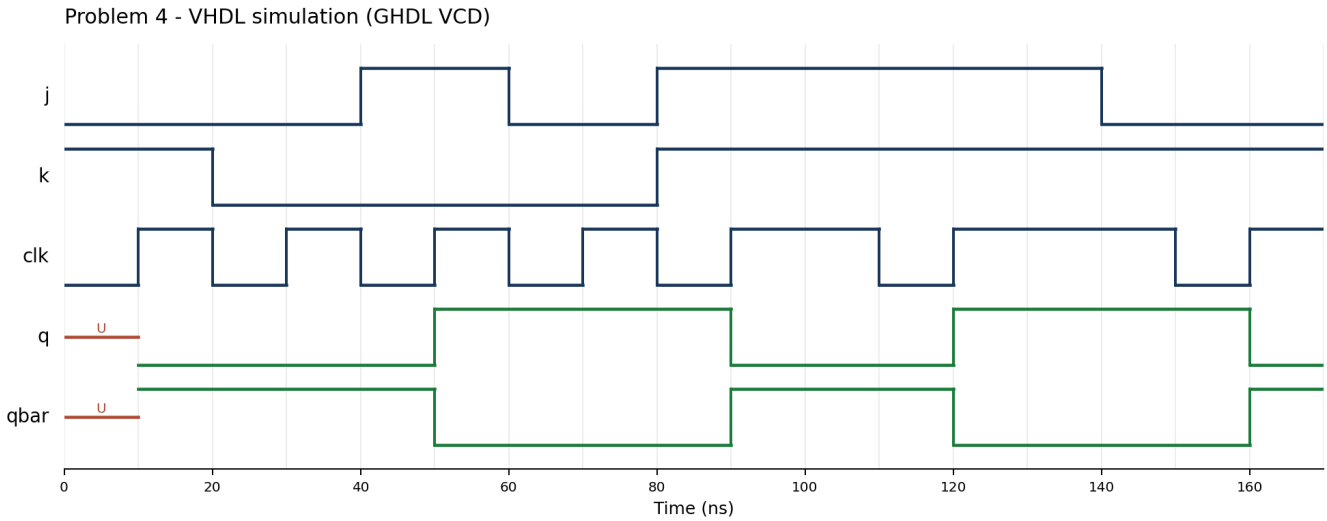


j=k=1의 상승 에지에서 1이 0으로 반전되었다.



그다음 상승 에지에서 0이 1로 반전되었다.

10 ns에서 Reset, 50 ns에서 Set을 확인하였다. j=k=0인 상승 에지에서는 이전 값이 유지되었다. j=k=1인 90 ns와 120 ns의 상승 에지에서 q가 각각 0, 1로 반전되었다. 그 뒤 clk를 계속 1로 유지한 구간에서는 추가 반전이 없었다.



파형은 GHDL이 기록한 VCD를 표시한 것이다. 각 입력 단계는 10 ns이며, 게이트 회로에도 같은 순서의 입력을 적용하였다.

4. RESULTS

목적
JK 입력 조합에 따른 네 가지 동작과 상승 에지에서만 이루어지는 출력 갱신을 확인한다.

회로 구성
JK 입력 논리 4개, 클록 반전 1개, 두 D Latch 10개를 합하여 NAND 게이트 15개를 사용하였다. q와 qbar를 JK 입력 논리에 되돌려 연결하였다.

동작 설명
j=k=0이면 다음 값은 현재 q와 같아 Hold가 된다. j=0, k=1이면 Reset, j=1, k=0이면 Set이 된다. j=k=1이면 다음 값은 qbar가 되어 Toggle이 일어난다. Master-Slave 구조를 사용하므로 높은 클록 레벨 동안 반복해서 반전되지 않는다. 초기 출력이 정해지지 않은 상태에서 Hold나 Toggle부터 시작하지 않고 Reset을 먼저 적용하였다. 이 회로는 단일 JK 래치가 아니므로 clk=1에서 피드백이 반복 반전되는 경로가 열리지 않는다.

초기화 단계 다음의 16개 검사에서 세 구현의 결과가 일치하였다. 두 번의 연속 상승 에지에서 Toggle을 확인하였으며, 클록을 1로 유지하거나 상승 에지 전에 입력을 바꾼 경우에는 q가 유지되었다.

Problem 5. D Flip-Flop with Asynchronous Set and Clear

Requirements

상승 에지 D Flip-Flop에 비동기 Set과 Clear를 추가한다. clr=1이면 클록과 관계없이 q=0, clr=0이고 set=1이면 q=1이 되어야 한다. 두 제어가 0일 때에는 일반적인 D Flip-Flop으로 동작하며, 두 제어가 동시에 1이면 Clear를 우선한다.

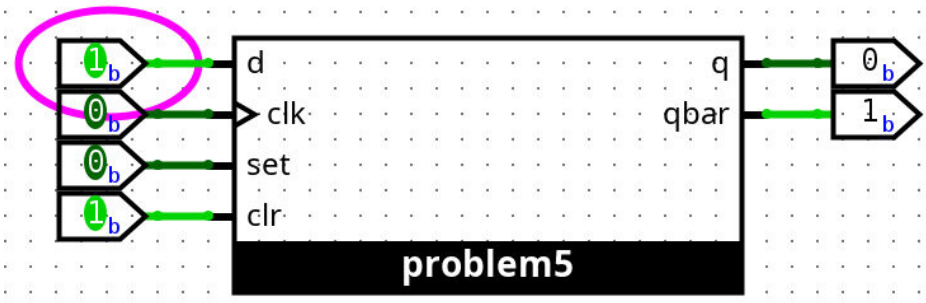
입력: d, clk, set, clr / 출력: q, qbar. 모든 핀은 1비트이며 Set과 Clear는 1에서 활성화된다.

1. VHDL code

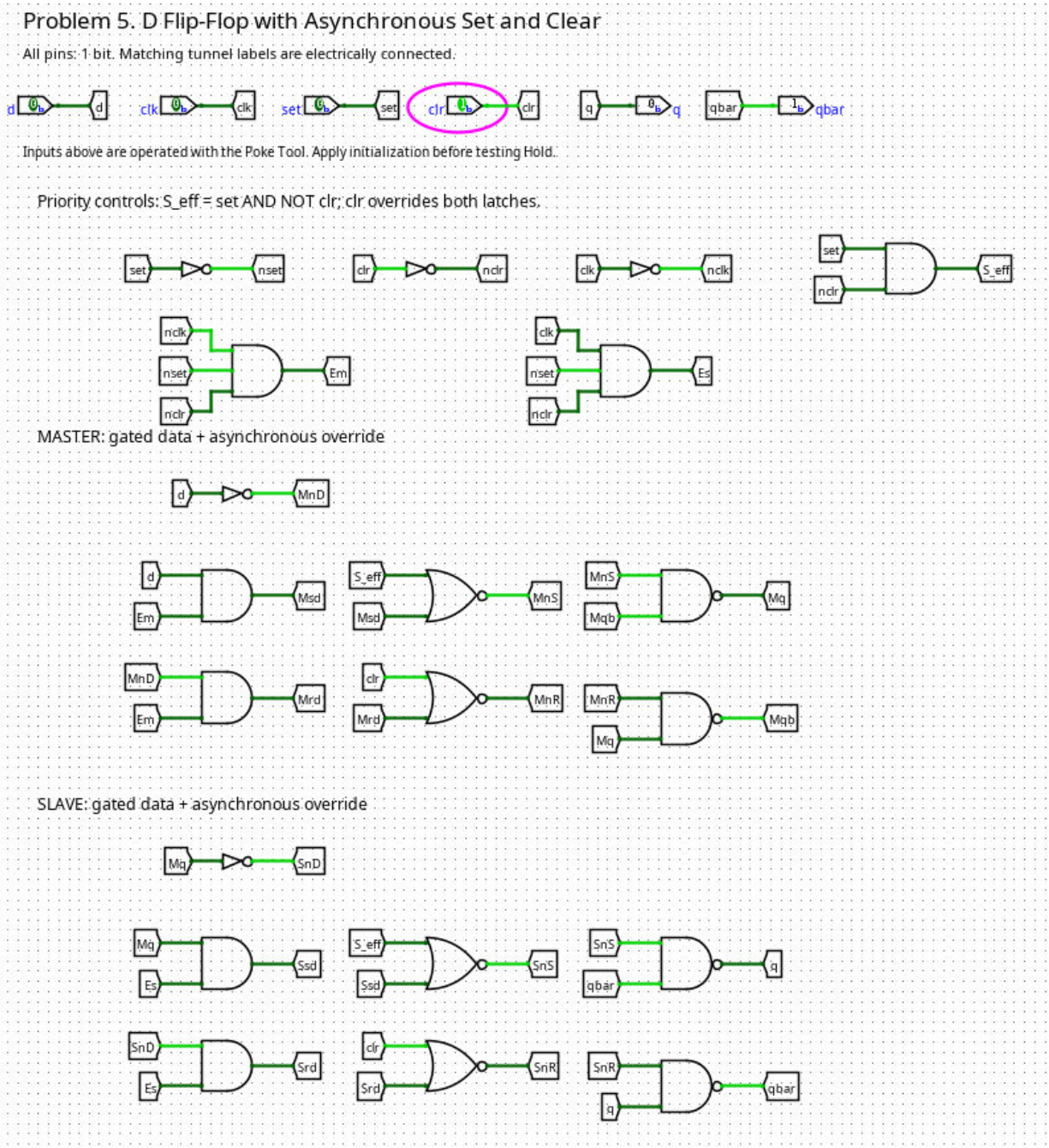
```
library ieee;
use ieee.std_logic_1164.all;
entity problem5 is
    port (
        d, clk, set, clr : in std_logic;
        q, qbar : out std_logic
    );
end problem5;
architecture behavioral of problem5 is
    signal qi : std_logic;
begin
    process(clk, set, clr)
    begin
        if clr = '1' then
            qi <= '0';
        elsif set = '1' then
            qi <= '1';
        elsif rising_edge(clk) then
            qi <= d;
        end if;
    end process;
    q <= qi;
    qbar <= not qi;
end behavioral;
```

2. Basic circuit

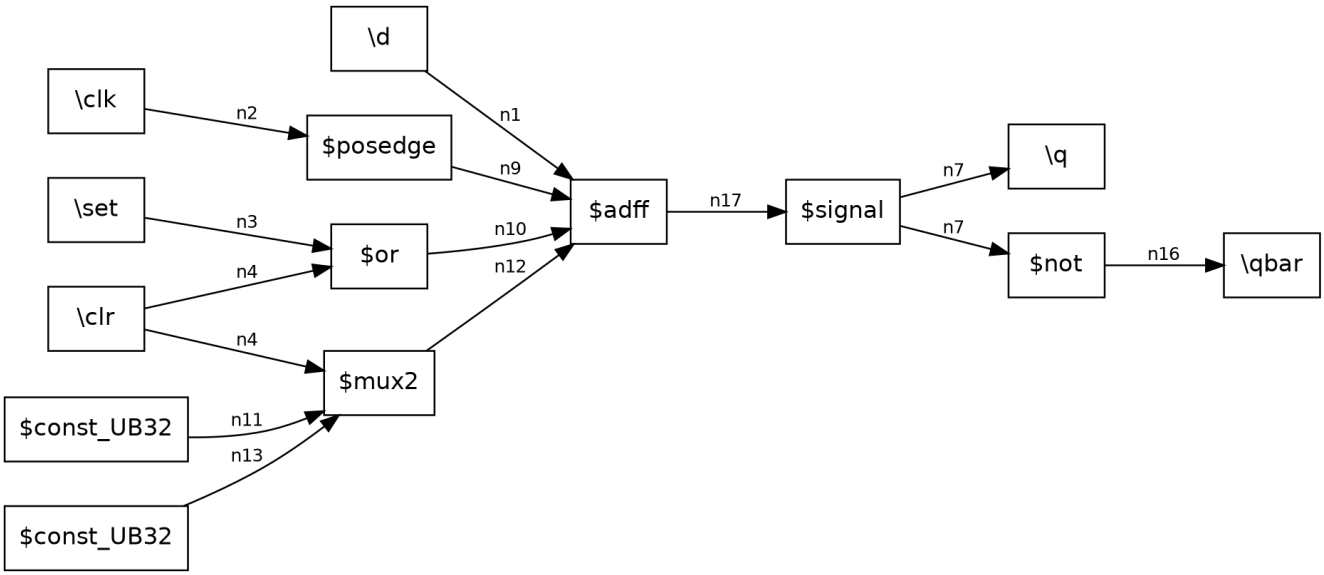
main에서 d, clk, set, clr을 problem5에 연결하였다. 모든 핀은 1비트이다. clr=1로 초기화한 후 clr=0으로 내려 시험하였다. set과 clr은 1일 때 활성화된다.



Master와 Slave 모두에 비동기 제어를 연결하였다. S_eff=set AND NOT clr로 Clear 우선순위를 만들고, Set 또는 Clear가 활성화되면 일반 데이터 경로를 막았다. 각 래치의 nS와 nR을 통해 저장된 상태를 직접 바꾸도록 하였다.

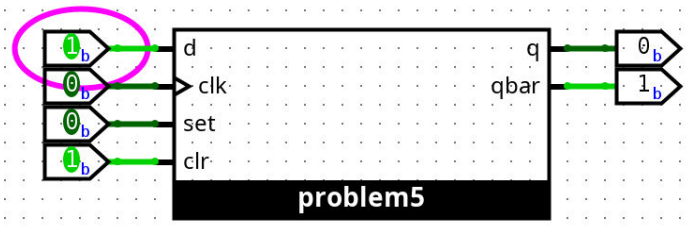


GHDL 합성 결과에는 비동기 제어를 갖는 저장 소자($adff)와 제어 논리가 생성되었다. clr이 1이면 비동기 저장값이 0으로 선택되어 Clear가 우선한다. 생성한 netlist에도 같은 입력을 적용하여 동작을 확인하였다.

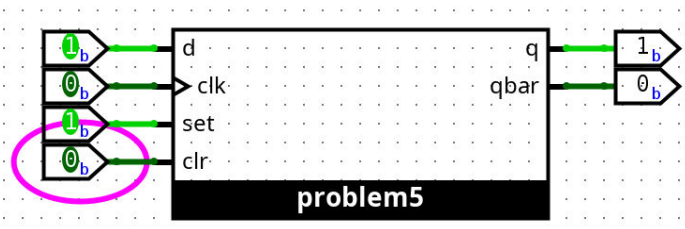


3. Simulation results

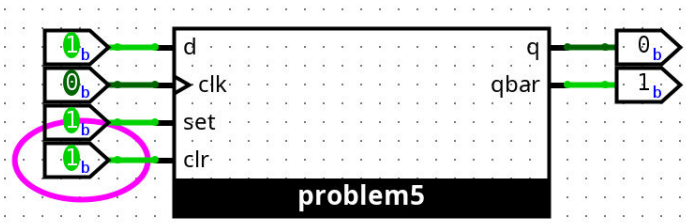
클록을 고정한 상태에서 Set과 Clear를 변경하여 비동기 동작을 확인하였다. 두 제어가 모두 1인 조건과 제어를 해제한 뒤의 정상 저장 동작도 확인하였다.



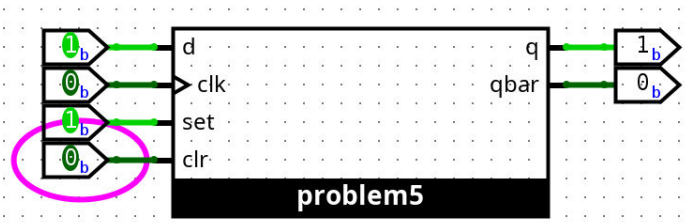
clk=0, clr=1에서 d=1이어도 q=0이었다.



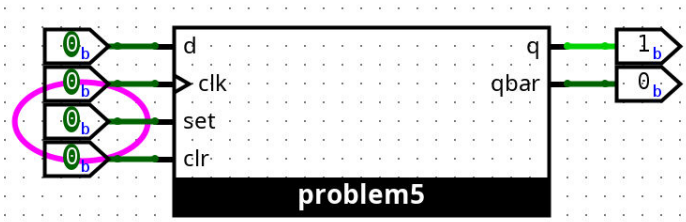
clk=0, set=1, clr=0에서 q=1이 되었다.



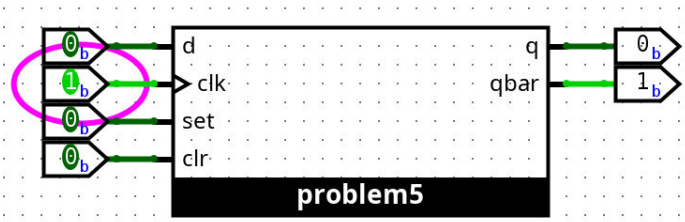
set=clr=1에서는 Clear가 우선하여 q=0이 되었다.



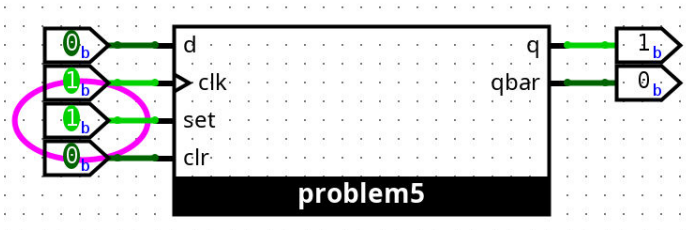
set=1을 유지하고 clr만 0으로 내리자 q=1이 되었다.



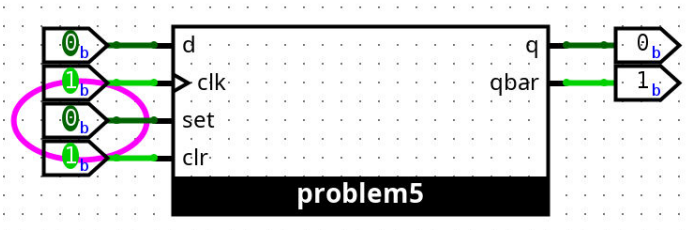
두 제어를 해제한 뒤 새 상승 에지 전까지 q=1을 유지하였다.



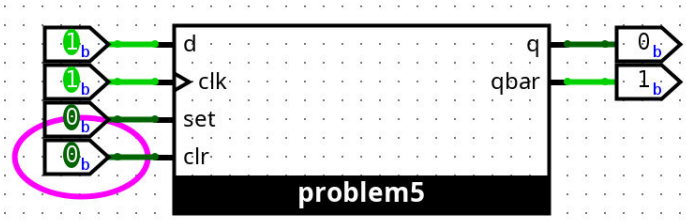
정상 모드의 상승 에지에서 d=0을 저장하였다.



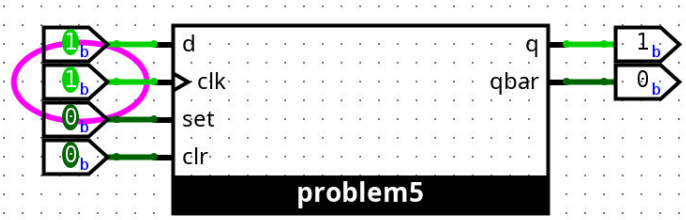
clk=1에서 상승 에지 없이 Set이 동작하였다.



clk=1에서 상승 에지 없이 Clear가 동작하였다.

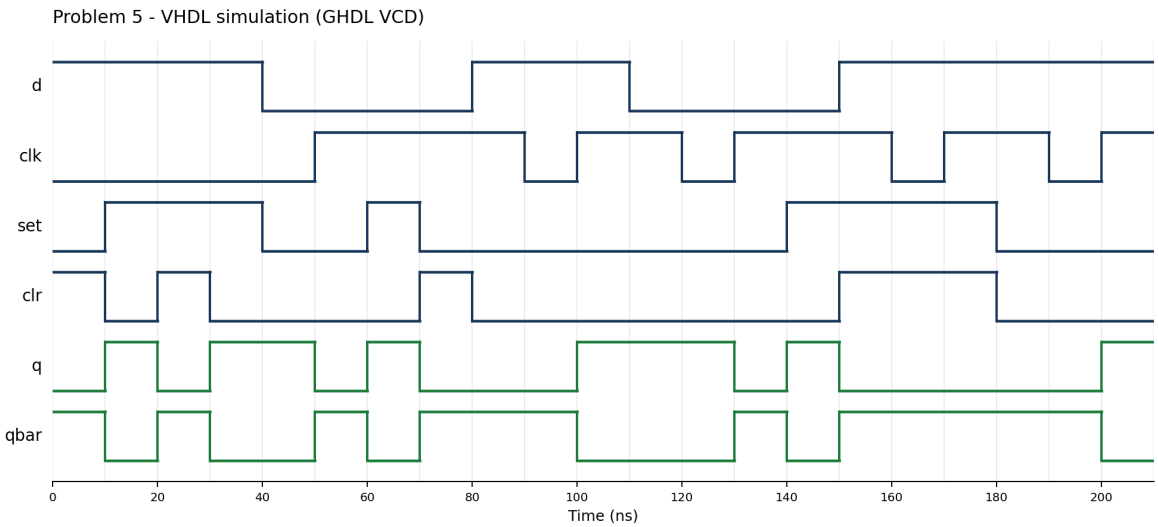


clk=1에서 두 제어를 해제해도 q=0을 유지하였다.



다음 상승 에지에서 d=1을 저장하였다.

clk=0인 10 ns에 Set을 주자 q=1이 되었고, 20 ns에 Clear도 1로 올리자 q=0이 되었다. 30 ns에는 Set을 유지한 채 Clear만 0으로 내려 q=1이 되는 것을 확인하였다. clk=1인 60 ns와 70 ns에서도 Set과 Clear가 각각 동작하였다. 두 제어가 0인 구간에서는 상승 에지에서만 d를 저장하였다.



파형은 GHDL이 기록한 VCD를 표시한 것이다. 각 입력 단계는 10 ns이며, 게이트 회로에도 같은 순서의 입력을 적용하였다.

4. RESULTS

목적

비동기 Set/Clear와 정상적인 상승 에지 저장 동작을 함께 확인하고, Clear 우선순위가 유지되는지 검사한다.

회로 구성

NAND로 구성한 두 래치에 AND, NOR, NOT 게이트로 비동기 입력 제어를 추가하였다. 완성된 회로는 기본 게이트 20개로 구성되며, 두 래치에 같은 Set과 Clear를 연결하였다.

동작 설명
clr=1이면 S_eff=0이고 nR이 0이 되어 q=0이 된다. clr=0, set=1이면 nS가 0이 되어 q=1이 된다. 두 제어가 0이면 Master는 NOT clk, Slave는 clk에 의해 동작하여 상승 에지에서 d를 저장한다. VHDL에서도 clr, set, rising_edge(clk)의 순서로 조건을 검사하였다.

21개 검사에서 세 구현의 결과가 일치하였다. 클록이 0인 경우와 1인 경우의 비동기 동작, 두 제어가 동시에 1인 경우, Clear만 해제하는 경우, 두 제어를 해제한 뒤의 유지와 정상 저장 동작을 확인하였다.

Overall Discussion

이번 실습에서는 피드백으로 상태를 저장하는 래치와 상승 에지에서 값을 저장하는 플립플롭을 비교하였다. D Latch는 en=1인 동안 입력을 따라가지만, D Flip-Flop과 JK Flip-Flop은 상승 에지에서만 출력이 갱신되었다. 비동기 Set과 Clear는 클록을 기다리지 않고 출력을 바꾸며, 두 제어를 함께 주면 Clear가 우선하였다.

순차회로는 이전 상태가 결과에 영향을 주므로 초기화와 입력 순서를 정해 두어야 했다. 특히 JK 회로는 단일 래치로 구성하면 Toggle 조건에서 반복 반전이 생길 수 있어 NAND 게이트로 만든 Master-Slave 구조를 사용하였다. 게이트 회로, VHDL 원본과 합성 netlist에서 총 64개의 판정 결과가 일치하였다.